

Лабораторная работа №13

Программирование в командном процессоре ОС UNIX. Ветвления и циклы

Иванова Анастасия Сергеевна

2026-05-08

Содержание I

1. Докладчик
2. Вводная часть
3. Выполнение лабораторной работы
4. Контрольные вопросы
5. Вывод

Раздел 1

1. Докладчик

1. Докладчик

- Иванова Анастасия Сергеевна



1. Докладчик

- Иванова Анастасия Сергеевна
- 1 курс группа НКАбд-07-25



1. Докладчик

- Иванова Анастасия Сергеевна
- 1 курс группа НКАбд-07-25
- Российский университет дружбы народов



1. Докладчик

- Иванова Анастасия Сергеевна
- 1 курс группа НКАбд-07-25
- Российский университет дружбы народов
- 1132250427@rudn.ru



Раздел 2

2. Вводная часть

- Ветвления и циклы – основа алгоритмизации в командных оболочках UNIX

- Ветвления и циклы – основа алгоритмизации в командных оболочках UNIX
- Позволяют создавать гибкие скрипты для автоматизации административных задач

- Ветвления и циклы – основа алгоритмизации в командных оболочках UNIX
- Позволяют создавать гибкие скрипты для автоматизации административных задач
- Умение обрабатывать ключи командной строки (getopts) необходимо для профессиональных скриптов

- Ветвления и циклы – основа алгоритмизации в командных оболочках UNIX
- Позволяют создавать гибкие скрипты для автоматизации административных задач
- Умение обрабатывать ключи командной строки (`getopts`) необходимо для профессиональных скриптов
- Анализ кода возврата программ и работа с `find/tar` расширяют возможности shell-программирования

Объект и предмет исследования

Объект исследования: - Командный процессор bash - Управляющие конструкции: if, case, for, while, until - Механизмы обработки аргументов и кодов завершения

Предмет исследования: - Разработка скриптов с использованием getopt s - Создание программы на C и анализ её кода завершения в оболочке - Циклическое создание и удаление файлов - Фильтрация файлов по времени изменения и архивация

- Изучить основы программирования в оболочке ОС UNIX

- Изучить основы программирования в оболочке ОС UNIX
- Научиться писать более сложные командные файлы с использованием логических управляющих конструкций и циклов

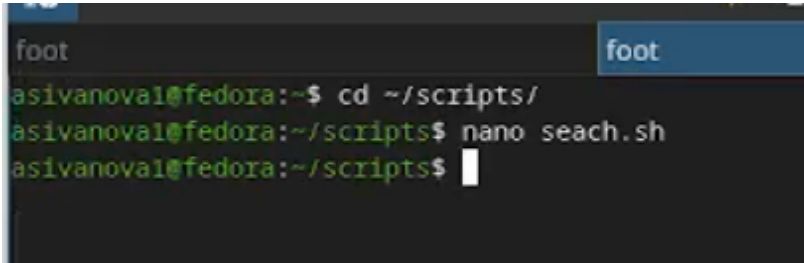
Раздел 3

3. Выполнение лабораторной работы

Задание 1. Скрипт с ключами getopt

Разработаем скрипт `search.sh`, который анализирует командную строку с ключами `-inputfile`, `-outputfile`, `-p`, `-C`, `-n` и выполняет поиск строк в указанном файле.

- 1 Создадим файл скрипта `search.sh` (рис. 2.1):



```
foot
asivanova1@fedora:~$ cd ~/scripts/
asivanova1@fedora:~/scripts$ nano seach.sh
asivanova1@fedora:~/scripts$
```

Рисунок 1: Создание файла `search.sh`

2 Листинг скрипта search.sh (рис. 2.2):

```
#!/bin/bash
```

```
input_file=""
```

```
output_file=""
```

```
pattern=""
```

```
case_sensitive="no"
```

```
number_lines="no"
```

```
usage() {
```

```
    echo "Использование: $0 -inputfile <файл> -p <шаблон> [-outputfile <файл>]
```

```
    exit 1
```

```
}
```

```
while getopts ":i:o:p:Cn" opt; do
```

```
    case $opt in
```

```
        i) input_file="$OPTARG" ;;
```

```
foot search.sh https://mail.yandex.ru/ru/
GNU nano 8.5
#!/bin/bash

input_file=""
output_file=""
pattern=""
case_sensitive="no"
number_lines="no"

usage() {
    echo "Использование: $0 -inputfile <файл> -p <шаблон> [-outputfile <файл>] [-C] [-n]"
    exit 1
}

while getopts ":i:o:p:Cn" opt; do
    case $opt in
        i) input_file="$OPTARG" ;;
        o) output_file="$OPTARG" ;;
        p) pattern="$OPTARG" ;;
        C) case_sensitive="yes" ;;
        n) number_lines="yes" ;;
        \?) echo "Неизвестный параметр: -$OPTARG" >&2; usage ;;
        :) echo "Параметр -$OPTARG требует аргумента" >&2; usage ;;
    esac
done

if [ -z "$input_file" ] || [ -z "$pattern" ]; then
    echo "Ошибка: необходимо указать -inputfile и -p"
    usage
fi

Поиск [Назад]:
```

- ③ Сделаем файл исполняемым и подготовим тестовый файл `test.txt` (рис. 2.3):

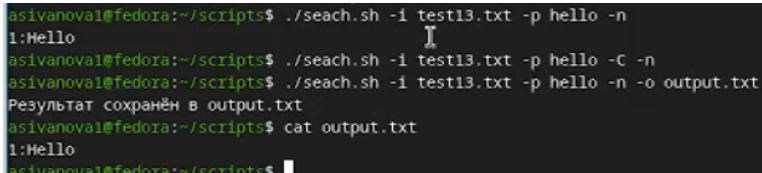
```
echo "Hello world" > test.txt  
echo "hello again" >> test.txt  
echo "HELLO upper" >> test.txt  
echo "goodbye" >> test.txt
```

```
asivanova1@fedora:~/scripts$ chmod +x seach.sh  
asivanova1@fedora:~/scripts$ echo "Hello" > test13.txt  
asivanova1@fedora:~/scripts$ echo "again" >> test13.txt  
asivanova1@fedora:~/scripts$ echo "bye" >> test13.txt
```

Рисунок 3: Создание test.txt

❷ Запустим скрипт с различными ключами (рис. 2.4):

```
./search.sh -i test.txt -p hello -n  
./search.sh -i test.txt -p hello -C -n  
./search.sh -i test.txt -p hello -n -o output.txt  
cat output.txt
```



```
asivanova1@fedora:~/scripts$ ./seach.sh -i test13.txt -p hello -n  
1:Hello  
asivanova1@fedora:~/scripts$ ./seach.sh -i test13.txt -p hello -C -n  
asivanova1@fedora:~/scripts$ ./seach.sh -i test13.txt -p hello -n -o output.txt  
Результат сохранён в output.txt  
asivanova1@fedora:~/scripts$ cat output.txt  
1:Hello  
asivanova1@fedora:~/scripts$
```

Рисунок 4: Результаты работы search.sh

Задание 2. Программа на С и анализ кода завершения

Создадим программу на С `checknum.c`, которая вводит число и возвращает код завершения в зависимости от его знака. Затем напишем скрипт, анализирующий этот код.

❶ Листинг `checknum.c` (рис. 2.5):

```
#include <stdio.h>
#include <stdlib.h>

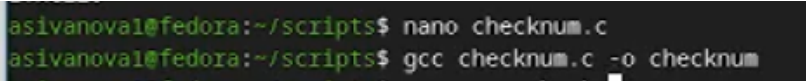
int main() {
    int n;
    printf("Введите число: ");
    scanf("%d", &n);
    if (n > 0) {
        printf("Число положительное\n");
        exit(1);
    } else if (n < 0) {
        printf("Число отрицательное\n");
    }
```

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n;
    printf("Введите число: ");
    scanf("%d", &n);
    if (n > 0) {
        printf("Число положительное\n");
        exit(1);
    } else if (n < 0) {
        printf("Число отрицательное\n");
        exit(2);
    } else {
        printf("Число равно нулю\n");
        exit(0);
    }
}
```

- 2 Скомпилируем программу (рис. 2.6):

```
gcc checknum.c -o checknum
```



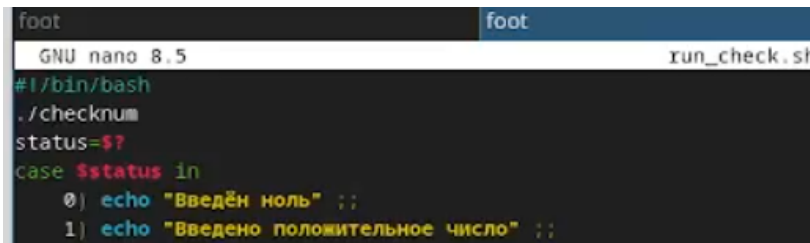
```
asivanova1@fedora:~/scripts$ nano checknum.c  
asivanova1@fedora:~/scripts$ gcc checknum.c -o checknum
```

A terminal window with a dark background. The prompt is 'asivanova1@fedora:~/scripts\$'. The first command is 'nano checknum.c' and the second is 'gcc checknum.c -o checknum'. The output of the second command is not visible.

Рисунок 6: Компиляция checknum.c

3 Создадим скрипт `run_check.sh` для анализа кода возврата (рис. 2.7):

```
#!/bin/bash
./checknum
status=$?
case $status in
    0) echo "Введён ноль" ;;
    1) echo "Введено положительное число" ;;
    2) echo "Введено отрицательное число" ;;
    *) echo "Неизвестный код завершения" ;;
esac
```



```
foot
GNU nano 8.5 run_check.sh
#!/bin/bash
./checknum
status=$?
case $status in
    0) echo "Введён ноль" ;;
    1) echo "Введено положительное число" ;;
```

❷ Сделаем файл исполняемым и протестируем на числах 8, -10, 0 (рис. 2.8 – 2.10):

```
./run_check.sh    # ввод 8  
./run_check.sh    # ввод -10  
./run_check.sh    # ввод 0
```

```
asivanova1@fedora:~/scripts$ chmod +x run_check.sh
asivanova1@fedora:~/scripts$ ./run_check
bash: ./run_check: Нет такого файла или каталога
asivanova1@fedora:~/scripts$ ./run_check.sh
Введите число: 8
Число положительное
Введено положительное число
```

Рисунок 8: Результат для 8


```
asivanova1@fedora:~/scripts$ ./run_check.sh  
Введите число: -10  
Число отрицательное  
Введено отрицательное число
```

Рисунок 9: Результат для -10

```
asivanova1@fedora:~/scripts$ ./run_check.sh
Введите число: 0
Число равно нулю
Введён ноль
```

Рисунок 10: Результат для 0

Задание 3. Создание и удаление нумерованных файлов

Разработаем скрипт `file_manager.sh`, который создаёт или удаляет файлы `1.tmp`, `2.tmp`, ..., `N.tmp` в зависимости от наличия ключа `-d`.

❶ Листинг `file_manager.sh` (рис. 2.11):

```
#!/bin/bash

if [ $# -lt 1 ]; then
    echo "Использование: $0 [-d] N"
    echo "  без -d: создаёт файлы 1.tmp .. N.tmp"
    echo "  с -d: удаляет файлы 1.tmp .. N.tmp"
    exit 1
fi

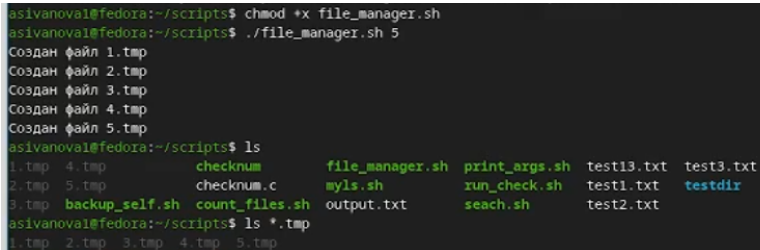
if [ "$1" = "-d" ]; then
    delete_mode=1
    N=$2
```

```
foot
GNU nano 8.5
file_manager.sh
if [ $delete_mode -eq 1 ]; then
    for i in $(seq 1 $N); do
        if [ -f "$i.tmp" ]; then
            rm "$i.tmp"
            echo "Удалён файл $i.tmp"
        else
            echo "Файл $i.tmp не существует"
        fi
    done
else
    for i in $(seq 1 $N); do
        touch "$i.tmp"
        echo "Создан файл $i.tmp"
    done
fi
```

Рисунок 11: Листинг file_manager.sh

② Сделаем файл исполняемым и создадим 5 файлов (рис. 2.12):

```
chmod +x file_manager.sh
./file_manager.sh 5
ls *.tmp
```

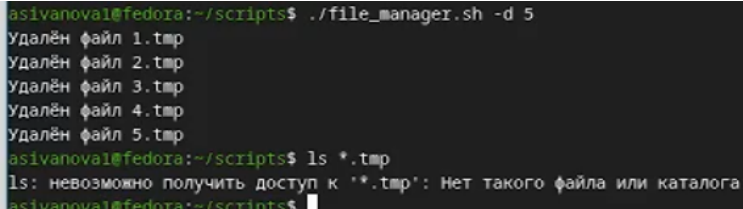


```
asivanova1@fedora:~/scripts$ chmod +x file_manager.sh
asivanova1@fedora:~/scripts$ ./file_manager.sh 5
Создан файл 1.tmp
Создан файл 2.tmp
Создан файл 3.tmp
Создан файл 4.tmp
Создан файл 5.tmp
asivanova1@fedora:~/scripts$ ls
1.tmp  4.tmp      checknum    file_manager.sh  print_args.sh  test13.txt  test3.txt
2.tmp  5.tmp      checknum.c  myls.sh          run_check.sh   test1.txt   testdir
3.tmp  backup_self.sh  count_files.sh  output.txt       seach.sh       test2.txt
asivanova1@fedora:~/scripts$ ls *.tmp
1.tmp 2.tmp 3.tmp 4.tmp 5.tmp
```

Рисунок 12: Создание файлов

- ③ Удалим созданные файлы (рис. 2.13):

```
./file_manager.sh -d 5  
ls *.tmp
```



The screenshot shows a terminal window with the following text:

```
asivanova1@fedora:~/scripts$ ./file_manager.sh -d 5  
Удалён файл 1.tmp  
Удалён файл 2.tmp  
Удалён файл 3.tmp  
Удалён файл 4.tmp  
Удалён файл 5.tmp  
asivanova1@fedora:~/scripts$ ls *.tmp  
ls: невозможно получить доступ к '*.tmp': Нет такого файла или каталога  
asivanova1@fedora:~/scripts$
```

Рисунок 13: Удаление файлов

Задание 4. Архивация свежих файлов

Разработаем скрипт `backup_recent.sh`, который архивирует только файлы, изменённые за последние 7 дней в указанной директории.

❶ Листинг `backup_recent.sh` (рис. 2.14):

```
#!/bin/bash

if [ $# -ne 1 ]; then
    echo "Использование: $0 <директория>"
    exit 1
fi

DIR="$1"

if [ ! -d "$DIR" ]; then
    echo "Ошибка: директория $DIR не существует"
    exit 1
```

```
root      root      https://ma
GNU nano 8.5      backup_recent.sh
#!/bin/bash

if [ $# -ne 1 ]; then
    echo "Использование: $0 <директория>"
    exit 1
fi

DIR="$1"

if [ ! -d "$DIR" ]; then
    echo "Ошибка: директория $DIR не существует"
    exit 1
fi

BACKUP_NAME="backup_$(date +%Y%m%d_%H%M%S).tar.gz"

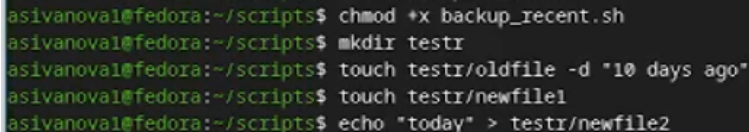
find "$DIR" -type f -mtime -7 -print | tar -czf "$BACKUP_NAME" -T -

echo "Архив создан: $BACKUP_NAME"
```

Рисунок 14: Листинг backup_recent.sh

- ② Сделаем файл исполняемым и подготовим тестовую директорию с файлами разной давности (рис. 2.15):

```
mkdir testdir  
touch testdir/oldfile -d "10 days ago"  
touch testdir/newfile1  
echo "test" > testdir/newfile2
```

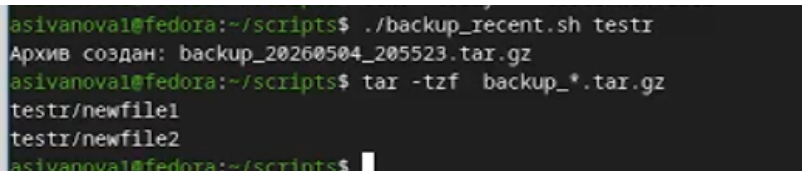
A terminal window with a dark background and green text. It shows a series of commands being executed to create a test directory and files. The prompt is 'asivanova1@fedora:~/scripts\$'. The commands are: 'chmod +x backup_recent.sh', 'mkdir testr', 'touch testr/oldfile -d "10 days ago"', 'touch testr/newfile1', and 'echo "today" > testr/newfile2'.

```
asivanova1@fedora:~/scripts$ chmod +x backup_recent.sh  
asivanova1@fedora:~/scripts$ mkdir testr  
asivanova1@fedora:~/scripts$ touch testr/oldfile -d "10 days ago"  
asivanova1@fedora:~/scripts$ touch testr/newfile1  
asivanova1@fedora:~/scripts$ echo "today" > testr/newfile2
```

Рисунок 15: Подготовка testdir

③ Запустим скрипт и проверим содержимое архива (рис. 2.16):

```
chmod +x backup_recent.sh  
./backup_recent.sh testdir  
tar -tzf backup_*.tar.gz
```



```
asivanova1@fedora:~/scripts$ ./backup_recent.sh testdir  
Архив создан: backup_20260504_205523.tar.gz  
asivanova1@fedora:~/scripts$ tar -tzf backup_*.tar.gz  
testdir/newfile1  
testdir/newfile2  
asivanova1@fedora:~/scripts$
```

Рисунок 16: Архивация и проверка

В архиве присутствуют только testdir/newfile1 и testdir/newfile2.

Раздел 4

4. Контрольные вопросы

4. Контрольные вопросы

❶ Каково предназначение команды `getopts`?

`getopts` используется для разбора опций командной строки в скриптах `bash`. Позволяет обрабатывать ключи с аргументами и без них, упрощая анализ параметров.

4. Контрольные вопросы

❶ Каково предназначение команды `getopts`?

`getopts` используется для разбора опций командной строки в скриптах `bash`. Позволяет обрабатывать ключи с аргументами и без них, упрощая анализ параметров.

❷ Какое отношение метасимволы имеют к генерации имён файлов?

Метасимволы (`*`, `?`, `[]`) служат для подстановки имён файлов (globbing) – замены шаблона на список существующих файлов.

4. Контрольные вопросы

❶ **Каково предназначение команды `getopts`?**

`getopts` используется для разбора опций командной строки в скриптах `bash`. Позволяет обрабатывать ключи с аргументами и без них, упрощая анализ параметров.

❷ **Какое отношение метасимволы имеют к генерации имён файлов?**

Метасимволы (`*`, `?`, `[]`) служат для подстановки имён файлов (globbing) – замены шаблона на список существующих файлов.

❸ **Какие операторы управления действиями вы знаете?**

`if`, `then`, `else`, `elif`, `fi`, `case`, `esac`, `for`, `while`, `until`, `break`, `continue`, `exit`.

4. Контрольные вопросы

❶ **Каково предназначение команды `getopts`?**

`getopts` используется для разбора опций командной строки в скриптах `bash`. Позволяет обрабатывать ключи с аргументами и без них, упрощая анализ параметров.

❷ **Какое отношение метасимволы имеют к генерации имён файлов?**

Метасимволы (`*`, `?`, `[]`) служат для подстановки имён файлов (globbing) – замены шаблона на список существующих файлов.

❸ **Какие операторы управления действиями вы знаете?**

`if`, `then`, `else`, `elif`, `fi`, `case`, `esac`, `for`, `while`, `until`, `break`, `continue`, `exit`.

❹ **Какие операторы используются для прерывания цикла?**

`break` – немедленный выход из цикла, `continue` – переход к следующей итерации.

4. Контрольные вопросы

❶ **Каково предназначение команды `getopts`?**

`getopts` используется для разбора опций командной строки в скриптах `bash`. Позволяет обрабатывать ключи с аргументами и без них, упрощая анализ параметров.

❷ **Какое отношение метасимволы имеют к генерации имён файлов?**

Метасимволы (`*`, `?`, `[]`) служат для подстановки имён файлов (globbing) – замены шаблона на список существующих файлов.

❸ **Какие операторы управления действиями вы знаете?**

`if`, `then`, `else`, `elif`, `fi`, `case`, `esac`, `for`, `while`, `until`, `break`, `continue`, `exit`.

❹ **Какие операторы используются для прерывания цикла?**

`break` – немедленный выход из цикла, `continue` – переход к следующей итерации.

❺ **Для чего нужны команды `false` и `true`?**

`true` всегда возвращает код 0 (истина), `false` – ненулевой код (ложь). Используются для организации бесконечных циклов или для задания условий.

4. Контрольные вопросы

❶ Каково предназначение команды `getopts`?

`getopts` используется для разбора опций командной строки в скриптах `bash`. Позволяет обрабатывать ключи с аргументами и без них, упрощая анализ параметров.

❷ Какое отношение метасимволы имеют к генерации имён файлов?

Метасимволы (`*`, `?`, `[]`) служат для подстановки имён файлов (globbing) – замены шаблона на список существующих файлов.

❸ Какие операторы управления действиями вы знаете?

`if`, `then`, `else`, `elif`, `fi`, `case`, `esac`, `for`, `while`, `until`, `break`, `continue`, `exit`.

❹ Какие операторы используются для прерывания цикла?

`break` – немедленный выход из цикла, `continue` – переход к следующей итерации.

❺ Для чего нужны команды `false` и `true`?

`true` всегда возвращает код 0 (истина), `false` – ненулевой код (ложь). Используются для организации бесконечных циклов или для задания условий.

❻ Что означает строка `if test -f man\${s}/\${i}.\${s}` в командном файле?

Проверяет существование файла, имя которого формируется из переменных `$s` и `$i`.

Обратные слесии экранируют метасимволы, чтобы они воспринимались буквально

4. Контрольные вопросы

❶ Каково предназначение команды `getopts`?

`getopts` используется для разбора опций командной строки в скриптах `bash`. Позволяет обрабатывать ключи с аргументами и без них, упрощая анализ параметров.

❷ Какое отношение метасимволы имеют к генерации имён файлов?

Метасимволы (`*`, `?`, `[]`) служат для подстановки имён файлов (globbing) – замены шаблона на список существующих файлов.

❸ Какие операторы управления действиями вы знаете?

`if`, `then`, `else`, `elif`, `fi`, `case`, `esac`, `for`, `while`, `until`, `break`, `continue`, `exit`.

❹ Какие операторы используются для прерывания цикла?

`break` – немедленный выход из цикла, `continue` – переход к следующей итерации.

❺ Для чего нужны команды `false` и `true`?

`true` всегда возвращает код 0 (истина), `false` – ненулевой код (ложь). Используются для организации бесконечных циклов или для задания условий.

❻ Что означает строка `if test -f man\${s}/\${i}.\${s}` в командном файле?

Проверяет существование файла, имя которого формируется из переменных `$s` и `$i`.

Обратные слесии экранируют метасимволы, чтобы они воспринимались буквально

Раздел 5

5. ВЫВОД

5. Вывод

Мы изучили основы программирования в оболочке ОС UNIX, научились писать более сложные командные файлы с использованием логических управляющих конструкций и циклов.

